

System logowania, Połączenie z bazą danych, Sesje

Wykłady 6

Bartosz Lauks

13 kwietnia 2026



① System logowania

② Sesje

③ System logowania (sesje)

- 1 System logowania
- 2 Sesje
- 3 System logowania (sesje)

Pseudo system logowania (bez sesji)

Kroki Pseudo system logowania

- Użytkownik dostaje się na naszą stronę internetową z formularzem logowania ([index.php](#)).
- Użytkownik wpisuje dane do formularza i zatwierdza go.
- Następuje przekierowanie danych do pliku, gdzie zostaną zwalidowane oraz sprawdzane czy konto użytkownika istnieje w serwisie ([login.php](#)).
- Połączenie do bazy danych ([config.php](#)).
- Wyświetlenie danych o użytkowniku na ekranie.

Pseudo system logowania (bez sesji)

Przetwarzanie requestu

- Kiedy użytkownik prześle dane do logowania odczytamy je po stornie serwera za pomocą **zmiennej globalnej** `$_POST`, tak aby odczytać przesłane dane z ciała żądania.
- Dane te będą zawierały się w tablicy asocjacyjnej, a kluczami tych wartości będą tagi **name** zadeklarowane w formularzu.

Pseudo system logowania (bez sesji)

Przechowywanie konta użytkownika

- Dane konta użytkownika zazwyczaj są przechowywane na serwerze, najczęściej wykorzystuje się do tego bazy danych.
- Baza danych zapewnia organizację i dostęp do danych oraz wysoką skalowalność i trwałość.
- W czasach, gdy technologie web dopiero raczkowały. Dane przechowywało się na przykład w plikach. Niestety takie rozwiązanie wiąże się z wieloma zagrożeniami.

Pseudo system logowania (bez sesji)

Tworzenie bazy danych na potrzeby wykładu

- Na potrzeby dzisiejszego wykładu skorzystamy z narzędzia [phpMyAdmin](#) i utworzymy prostą bazę danych z jedną tabelą o nazwie **user**.
- Tabela będzie zawierała następujące pola:
 - **id INT(11) A__I** – klucz podstawowy, unikalny dla każdego rekordu,
 - **name VARCHAR(100)** – nazwa użytkownika,
 - **password VARCHAR(100)** – hasło użytkownika,
 - **created_at TIMESTAMP** – opcjonalna kolumna informująca o dacie utworzenia rekordu.
- Ponieważ dziś nie skupiamy się na implementacji systemu logowania, tabelę uzupełnimy ręcznie w zakładce **Wstaw**, dodając przykładowe dane użytkownika.

Backup DB: [/lecture6/lecture6.sql](#)

Table name: Add column(s)

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index	
id Pick from Central Columns	INT	11	None			☐	PRIMARY	☒
name Pick from Central Columns	VARCHAR	100	None			☐	---	☐
password Pick from Central Columns	VARCHAR	100	None			☐	---	☐
created_at Pick from Central Columns	TIMESTAMP		CURRENT_TIME			☐	---	☐

Rysunek 1: Tworzenie tablicy user w PHPMyAdmin

Pseudo system logowania (bez sesji)

Dane do bazy danych

- Do połączenia z bazą danych musimy znać:
 - **adres IP** - adres hosta na którym znajduje się aplikacja bazy danych. W naszym przypadku będzie to **localhost** ponieważ uruchamiamy ją za pomocą **XAMPP**, zatem lokalnie
 - **nazwa bazy danych** - do której chcemy mieć dostęp.
 - **nazwa i użytkownika** - bazy danych.
- Dane te znajdziemy w **phpmyadmin** w zakładce **uprawnienia**.
- Ponieważ każde wywołanie zapytania (**query**) do bazy danych będzie wymagało wprowadzenie tych danych. Stworzymy plik, który będzie je przechowywał i w razie potrzeby dołączał do skryptów.

</lecture6/config.php>

Pseudo system logowania (bez sesji)

Bezpieczeństwo plików konfiguracyjnych

- W obecnej formie plik nie jest zabezpieczony.
- Oznacza to, że zwykły użytkownik może uzyskać do niego dostęp, podając bezpośrednią ścieżkę (URL).
- Pomimo tego, że kod PHP nie jest widoczny po stronie klienta, należy blokować dostęp do plików zawierających wrażliwe dane. W przypadku błędnej konfiguracji ich zawartość może zostać ujawniona.
- Istnieje wiele metod zabezpieczania takich plików.
- Zaleca się stosowanie kilku mechanizmów jednocześnie, aby w przypadku dezaktywacji jednego z nich nie utracić ochrony danych.
- Obecnie możemy zablokować dostęp do pliku [config.php](#) za pomocą pliku **.htaccess** lub przenieść go poza katalog udostępniany przez serwer (np. **htdocs**).

Pseudo system logowania (bez sesji)

Dołączanie plików w php (require & include)

- W wcześniej zostało wspomnianie że plik `config.php` będzie importowany ("doklejany") do skryptu w którym będą potrzebne dane logowania.
- Aby zrobić PHP udostępnia cztery funkcje:
 - **include** - Dołącza wskazany plik. Jeśli plik nie istnieje pojawi się ostrzeżenie (warning), ale skrypt będzie działał dalej.
 - **include_once** - Działa tak samo jak **include**, ale sprawdza, czy plik nie został już wcześniej dołączony.
 - **require** - Również dołącza plik, ale jeśli pliku nie ma przerywa wykonywanie skryptu (fatal error).
 - **require_once** - Sprawdza, czy plik nie był już dołączony, a jeśli nie – dołącza. Jeśli plik nie istnieje zatrzyma działanie skryptu.

Pseudo system logowania (bez sesji)

Połączenie z bazą danych

- W PHP mamy do dyspozycji dwie klasy do połączenia z bazą danych **MySQL**. Jest to: **mysqli** oraz **PDO**
- W dzisiejszym wykładzie skorzystamy z **mysqli**.
- Do konstruktora klasy musimy przekazać dane z [config.php](#), aby ustawić połączenie.
- Dopiero później wykonamy zapytanie SQL do bazy danych za pomocą metody **query()**.
- Następnie sprawdzimy czy przekazane dane przez użytkownika są prawidłowe, jeśli tak wyświetlimy jego imię.
- Opcjonalnie możemy użyć metody **header('Location: home.php')**, aby przekierować użytkownika na inny adres URL.

① System logowania

② Sesje

③ System logowania (sesje)

Sesje w PHP

- Aplikacje webowe są bezstanowe, co oznacza, że użytkownik nie jest „zapamiętywany” po otrzymaniu odpowiedzi serwera (response) na jego żądanie.
- Dlatego wszystkie zmienne zadeklarowane w poprzednim skrypcie są usuwane z pamięci po wysłaniu odpowiedzi **HTTP** do klienta, a co za tym idzie serwis nie pamięta że udało nam się zalogować.
- Aby poradzić sobie z tym problemem, PHP oferuje prosty mechanizm zarządzania sesjami, oparty na identyfikatorze **sesji**.
- Identyfikator sesji jest przechowywany w ciasteczku (**cookie**) na komputerze użytkownika.
- Przy każdym żądaniu (request) użytkownika PHP może przypisać je do odpowiedniej sesji, odczytując identyfikator.

Sesje a PHPSESSID

- **PHPSESSID** to domyślny identyfikator sesji wykorzystywany przez PHP w celu powiązania konkretnej sesji użytkownika z jego aktywnością na stronie.
- Kiedy użytkownik po raz pierwszy odwiedza stronę, PHP tworzy unikalny identyfikator sesji **PHPSESSID**.
- Ten identyfikator jest zapisywany w ciasteczku (cookie) przeglądarki użytkownika.
- Przy każdym kolejnym żądaniu (np. przejściu na inną stronę na tej samej witrynie), przeglądarka użytkownika wysyła ciasteczko **PHPSESSID** z powrotem do serwera. Dzięki temu serwer może powiązać to żądanie z odpowiednią sesją użytkownika

Tworzenie PHPSESSID przez przeglądarkę

- Dodawanie identyfikatora sesji można zaobserwować przy użyciu narzędzi **DevTools**.
- Podczas tworzenia sesji PHP dodaje do odpowiedzi serwera nagłówek **Set-Cookie**.
- Na jego podstawie przeglądarka wie, że powinna zapisać w swojej pamięci ciasteczko (cookie) o wartości określonej w nagłówku.
- Po zapisaniu ciasteczka przeglądarka automatycznie dołącza do każdego żądania wysyłanego do serwera nagłówek **Cookie**, w którym przekazywany jest identyfikator **PHPSESSID**.
- Dzięki temu serwer może zidentyfikować użytkownika i powiązać go z odpowiednią sesją przechowywaną po swojej stronie.

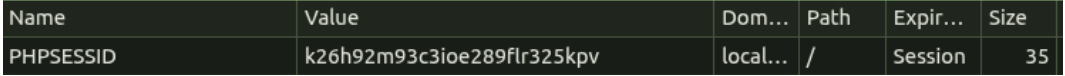
Set-CookiePHPSESSID=k26h92m93c3ioe289flr325kpv; path=/


Rysunek 2: Nagłówek odpowiedzi HTTP serwera nakazujący przeglądarce utworzenie cistka

CookiePHPSESSID=k26h92m93c3ioe289flr325kpv


Rysunek 3: Nagłówek żądania HTTP przeglądarki przekazujący serwerowi dane z cistek

Name	Value	Dom...	Path	Expir...	Size
PHPSESSID	k26h92m93c3ioe289flr325kpv	local...	/	Session	35



Rysunek 4: Widok danych cistka witryny (DevTools → Application → Cookies)

Tworzenie sesji w PHP

- Aby rozpocząć sesję w PHP, używamy funkcji `session_start()`
- Funkcja ta musi być wywołana na początku każdego skryptu, przed jakimkolwiek wysyłaniem danych do przeglądarki

Przechowywanie danych w sesji

- Po wywołaniu `session_start()` możemy przechowywać dane w zmiennych sesyjnych. Zmienna sesyjna to po prostu element w tablicy `$_SESSION`.

Sesje

Dostęp do danych sesji i ich usuwanie

- Aby odczytać dane zapisane w sesji, również używamy tablicy `$_SESSION`.
- Możemy usunąć pojedyncze elementy z sesji, używając funkcji `unset()`.
- Aby usunąć wszystkie dane sesji, możemy użyć funkcji `session_unset()`.

Zakończenie sesji

- Aby zakończyć sesję i usunąć wszystkie dane przechowywane w sesji, należy wywołać funkcję `session_destroy()`.
- Należy jednak pamiętać, że `session_destroy()` nie usuwa zmiennych sesyjnych w bieżącej sesji. Jeśli chcemy natychmiastowo wyczyścić zmienne sesyjne, możemy wywołać również `session_unset()`.

Przykłady: </lecture6/session-test/>

① System logowania

② Sesje

③ System logowania (sesje)

System logowania (sesje)

Dodanie sesji do systemu logowania

- Aby w naszym systemie logowania działały sesje, musimy w każdym pliku **.php** na początku wywołać funkcję `session_start()`. Dzięki temu sesja będzie dostępna w danym skrypcie.
- W ten sposób każdy użytkownik otrzymuje unikalny dostęp do zmiennej globalnej `$_SESSION`, która jest współdzielona pomiędzy skryptami korzystającymi z sesji.
- Po prawidłowym zalogowaniu ustawimy w niej zmienne informujące system o:
 - `$_SESSION['name'] = $name` – nazwe użytkownika,
 - `$_SESSION['logged'] = true` – określającej, czy użytkownik jest zalogowany.

</lecture6/login.php>

System logowania (sesje)

Zwracanie błędów formularza za pomocą sesji

- W chwili obecnej jeśli użytkownik poda nieprawidłowe dane podczas logowania wyświetli mu się na ekranie fraza "**Bad pass**" oraz utknie pod adresem URL </lecture6/login.php>.
- Dzięki implementacji sesji będziemy mogli wymusić na użytkownikowi ponowne przesłanie formularza oraz zwrócić informację o błędzie.

</lecture6/login.php>

</lecture6/index.php>

System logowania (sesje)

Zabezpieczanie URL

- Obecnie w naszym systemie istnieje luka.
- Użytkownik może pominąć logowanie i od razu odwołać się do pod adres </lecture6/home.php>.
- Doprowadzi to do błędu ponieważ skrypt będzie chciał uzyskać dostęp do niezadeklarowanej zmiennej `$name`.
- Aby się przed tym zabezpieczyć wykorzystamy jedną z technik, która polega na sprawdzaniu zmiennej utworzonej przy procesie logowania.
- Dzięki temu sprawdzimy czy użytkownik rzeczywiście jest zalogowany. W przeciwnym razie PHP wyśle go do formularza logowania.

</lecture6/home.php>

System logowania (sesje)

Cykl życia sesji

- Czas trwania sesji w PHP jest domyślnie określony przez wartość ustawioną w pliku konfiguracyjnym `php.ini` w parametrze `session.gc_maxlifetime`
- Określa maksymalny czas życia sesji w sekundach (domyślnie **1440 sekund**, czyli **24 minuty**). Po tym czasie sesja wygasa, a dane sesji są usuwane.
- Jeśli chcemy zmienić czas trwania sesji, możemy ustawić odpowiednią wartość w pliku `php.ini` lub bezpośrednio w skrypcie PHP za pomocą `ini_set('session.gc_maxlifetime', 3600);`

System logowania (sesje)

Wylogowywanie użytkownika

- Oczywiście każdy system powinien posiadać funkcjonalność wylogowywania użytkownika.
- Jeśli chodzi o jego implementację polega ona na wykorzystaniu funkcji sesji `session_unset()`, która usuwa wszystkie zmienne sesyjne.
- Przez zaimplementowane wcześniej zabezpieczenia bez zmiennej sesyjnej `$_SESSION['logged']` użytkownik utraci dostęp do zabezpieczonych podstron.

</lecture6/logout.php>

Dziękuję za uwagę !

Bartosz Lauks